EX. NO: 13 MINI PROJECT

DATE: **05/09/2026** **BANKING ACCOUN AND TRANSACTION MANAGEMENT**

Aim

To develop a Banking Account and Transaction Management System using SQL and PL/SQL/SQLite to store, manage, process, and analyze banking transaction records such as deposits, withdrawals, transaction amounts, and account balances.

Software Used

- **• GitHub Codespaces**
- **• SQLite**
- **• Oracle Database**
- **• SQL**
- **• PL/SQL**

Description

The Banking Account and Transaction Management System is developed to manage bank account transactions and customer banking activities. The system stores account details, transaction dates, transaction types, and transaction amounts.

SQL queries are used to retrieve and analyze transaction information, calculate total deposits and withdrawals, identify customers with the highest transaction amounts, display transactions exceeding ₹50,000, and calculate monthly transaction totals.

PL/SQL procedures and functions are used to process deposits and withdrawals, validate transaction amounts, retrieve the current account balance, and generate monthly account statements with appropriate validation and error handling.

Database Tables

1. BANK_TRANSACTIONS

Column Name	Data Type	Constraint
RANSACTION_ID	NTEGER	RIMARY KEY
CCOUNT_ID	NTEGER	OT NULL
USTOMER_NAME	ARCHAR(50)	OT NULL
RANSACTION_DATE	ATE/TEXT	OT NULL
RANSACTION_TYPE	ARCHAR(20)	OT NULL

MOUNT	ECIMAL(10,2)	OT NULL

Table Create

```
DROP TABLE IF EXISTS BANK_TRANSACTIONS;


CREATE TABLE BANK_TRANSACTIONS (
   transaction_id INTEGER PRIMARY KEY,
   account_id INTEGER NOT NULL,
   customer_name TEXT NOT NULL,
   transaction_date TEXT NOT NULL,
   transaction_type TEXT NOT NULL CHECK(transaction_type IN ('DEPOSIT',
'WITHDRAWAL')),
   amount REAL NOT NULL CHECK(amount > 0)
);
```

25 Records Insert

```
INSERT INTO BANK_TRANSACTIONS
(transaction_id, account_id, customer_name, transaction_date, transaction_type, amount)
VALUES
(1, 1001, 'Arun',   '2026-01-05', 'DEPOSIT',    50000),
(2, 1001, 'Arun',   '2026-01-10', 'WITHDRAWAL', 10000),
(3, 1001, 'Arun',   '2026-01-20', 'DEPOSIT',    30000),
(4, 1001, 'Arun',   '2026-02-05', 'DEPOSIT',    45000),
(5, 1001, 'Arun',   '2026-02-15', 'WITHDRAWAL', 15000),

(6, 1002, 'Priya',  '2026-01-03', 'DEPOSIT',    70000),
(7, 1002, 'Priya',  '2026-01-12', 'WITHDRAWAL', 20000),
(8, 1002, 'Priya',  '2026-01-25', 'DEPOSIT',    25000),
```

(9, 1002, 'Priya', '2026-02-08', 'DEPOSIT', 60000),

(10, 1002, 'Priya', '2026-02-20', 'WITHDRAWAL', 10000),

(11, 1003, 'Karthik', '2026-01-02', 'DEPOSIT', 40000),

(12, 1003, 'Karthik', '2026-01-15', 'WITHDRAWAL', 5000),

(13, 1003, 'Karthik', '2026-01-28', 'DEPOSIT', 55000),

(14, 1003, 'Karthik', '2026-02-10', 'WITHDRAWAL', 20000),

(15, 1003, 'Karthik', '2026-02-25', 'DEPOSIT', 30000),

(16, 1004, 'Divya', '2026-01-07', 'DEPOSIT', 90000),

(17, 1004, 'Divya', '2026-01-18', 'WITHDRAWAL', 25000),

(18, 1004, 'Divya', '2026-02-03', 'DEPOSIT', 35000),

(19, 1004, 'Divya', '2026-02-17', 'WITHDRAWAL', 10000),

(20, 1004, 'Divya', '2026-02-28', 'DEPOSIT', 50000),

(21, 1005, 'Rahul', '2026-01-09', 'DEPOSIT', 30000),

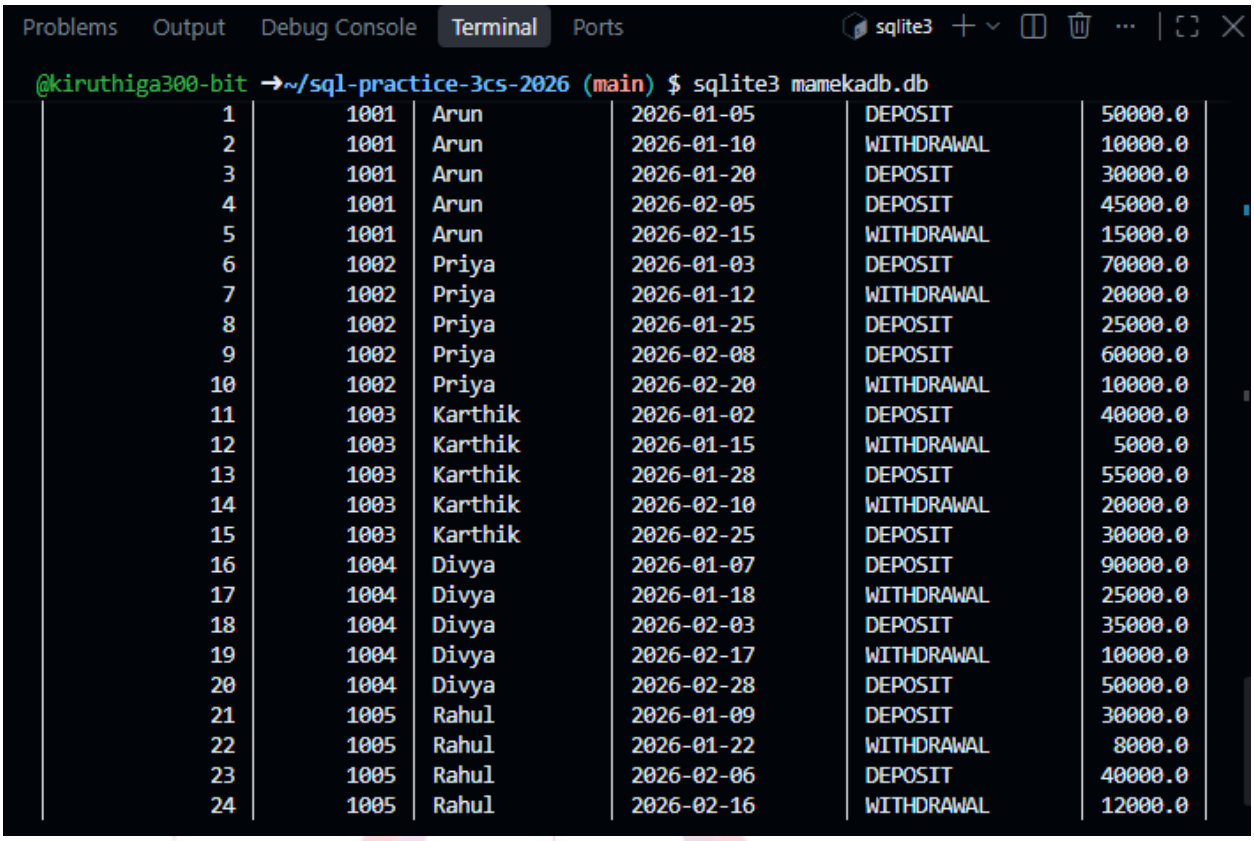
(22, 1005, 'Rahul', '2026-01-22', 'WITHDRAWAL', 8000),

(23, 1005, 'Rahul', '2026-02-06', 'DEPOSIT', 40000),

(24, 1005, 'Rahul', '2026-02-16', 'WITHDRAWAL', 12000),

(25, 1005, 'Rahul', '2026-02-26', 'DEPOSIT', 25000);

Output:



SQL Query 1

Display all transactions for a selected bank account

SELECT *

FROM BANK_TRANSACTIONS

WHERE account_id = 1001

ORDER BY transaction_date;

Output:

```
sqlite> SELECT *
FROM BANK_TRANSACTIONS
WHERE account_id = 1001
ORDER BY transaction_date;
```

transaction_id	account_id	customer_name	transaction_date	transaction_type	amount
1	1001	Arun	2026-01-05	DEPOSIT	50000.0
2	1001	Arun	2026-01-10	WITHDRAWAL	10000.0
3	1001	Arun	2026-01-20	DEPOSIT	30000.0
4	1001	Arun	2026-02-05	DEPOSIT	45000.0
5	1001	Arun	2026-02-15	WITHDRAWAL	15000.0

```
sqlite>
```

SQL Query 2

Calculate total deposit and withdrawal amounts

```
SELECT
   SUM(CASE
      WHEN transaction_type = 'DEPOSIT'
      THEN amount ELSE 0
   END) AS total_deposits,

   SUM(CASE
      WHEN transaction_type = 'WITHDRAWAL'
      THEN amount ELSE 0
   END) AS total_withdrawals
FROM BANK_TRANSACTIONS;
```

Output:

```
sqlite> SELECT
    SUM(CASE
        WHEN transaction_type = 'DEPOSIT'
        THEN amount ELSE 0
    END) AS total_deposits,

    SUM(CASE
        WHEN transaction_type = 'WITHDRAWAL'
        THEN amount ELSE 0
    END) AS total_withdrawals
FROM BANK_TRANSACTIONS;
```

total_deposits	total_withdrawals
675000.0	135000.0

```
sqlite>
```

SQL Query 3

Identify customers with highest total transaction amount

```
SELECT
    customer_name,
    account_id,
    SUM(amount) AS total_transaction_amount
FROM BANK_TRANSACTIONS
GROUP BY account_id, customer_name
ORDER BY total_transaction_amount DESC
LIMIT 1;
```

Output:

```
sqlite> SELECT
    customer_name,
    account_id,
    SUM(amount) AS total_transaction_amount
FROM BANK_TRANSACTIONS
GROUP BY account_id, customer_name
ORDER BY total_transaction_amount DESC
LIMIT 1;
```

customer_name	account_id	total_transaction...
Divya	1004	210000.0

```
sqlite>
```

SQL Query 4

Display transactions exceeding ₹50,000

```
SELECT *
FROM BANK_TRANSACTIONS
WHERE amount > 50000
ORDER BY amount DESC;
```

Output:

```
sqlite> SELECT *
FROM BANK_TRANSACTIONS
WHERE amount > 50000
ORDER BY amount DESC;
```

transaction_id	account_id	customer_name	transaction_date	transaction_type	amount
16	1004	Divya	2026-01-07	DEPOSIT	90000.0
6	1002	Priya	2026-01-03	DEPOSIT	70000.0
9	1002	Priya	2026-02-08	DEPOSIT	60000.0
13	1003	Karthik	2026-01-28	DEPOSIT	55000.0

```
sqlite>
```

SQL Query 5

Calculate monthly transaction totals

SELECT

strftime('%Y-%m', transaction_date) AS month,

SUM(amount) AS monthly_total

FROM BANK_TRANSACTIONS

GROUP BY strftime('%Y-%m', transaction_date)

ORDER BY month;

Output:

```
sqlite> SELECT
    strftime('%Y-%m', transaction_date) AS month,
    SUM(amount) AS monthly_total
FROM BANK_TRANSACTIONS
GROUP BY strftime('%Y-%m', transaction_date)
ORDER BY month;
```

month	monthly_total
2026-01	458000.0
2026-02	352000.0

```
sqlite>
```

PL/SQL Task 1 → SQLite Equivalent

Create a procedure to process a deposit or withdrawal after validating the transaction amount.

```
CREATE TRIGGER validate_transaction_amount

BEFORE INSERT ON BANK_TRANSACTIONS

FOR EACH ROW

WHEN NEW.amount <= 0

BEGIN

   SELECT RAISE(ABORT, 'Transaction amount must be greater than zero');

END;
```

Output:

```
sqlite> INSERT INTO BANK_TRANSACTIONS
(account_id, customer_name, transaction_date, transaction_type, amount)
VALUES
(1001, 'Arun', '2026-03-02', 'WITHDRAWAL', 999999);
Error near line 12: Insufficient funds
sqlite>
```

SELEC

T

PL/SQL Task 2 → Current Balance

Create a function to retrieve the current balance of an account.

```
SELECT

   account_id,

   customer_name,

   SUM(

      CASE

         WHEN transaction_type = 'DEPOSIT'
```

THEN amount

ELSE -amount

END

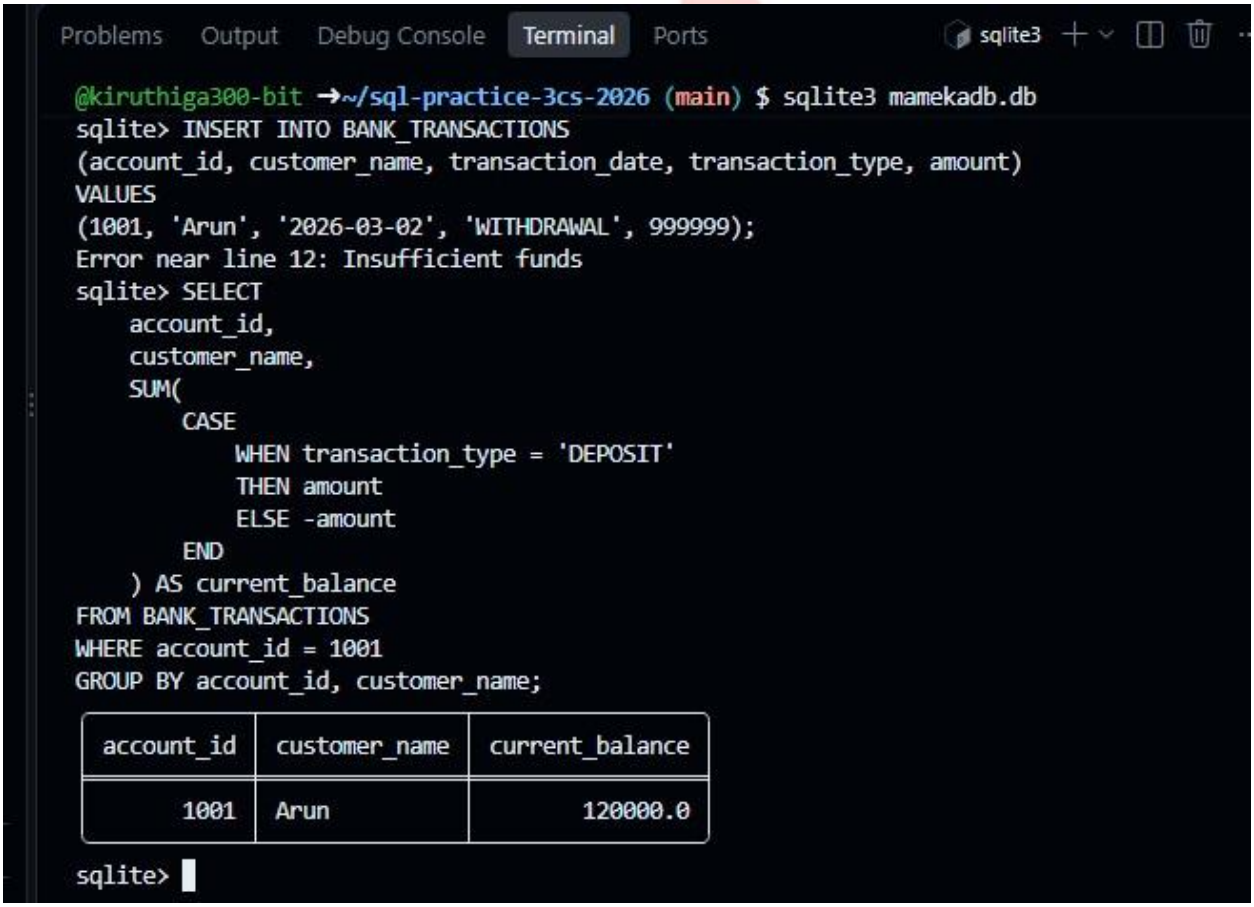
) AS current_balance

FROM BANK_TRANSACTIONS

WHERE account_id = 1001

GROUP BY account_id, customer_name;

output



PL/SQL Task 3 → Monthly Account Statement

Create a procedure to generate a monthly account statement, handling invalid account IDs and insufficient funds.

SELECT

transaction_id,

```
    account_id,
    customer_name,
    transaction_date,
    transaction_type,
    amount
FROM BANK_TRANSACTIONS
WHERE account_id = 1001
AND strftime('%Y-%m', transaction_date) = '2026-02'
ORDER BY transaction_date;
```

Output:

```
sqlite> SELECT
    transaction_id,
    account_id,
    customer_name,
    transaction_date,
    transaction_type,
    amount
FROM BANK_TRANSACTIONS
WHERE account_id = 1001
AND strftime('%Y-%m', transaction_date) = '2026-02'
ORDER BY transaction_date;
```

transaction_id	account_id	customer_name	transaction_date	transaction_type	amount
4	1001	Arun	2026-02-05	DEPOSIT	45000.0
5	1001	Arun	2026-02-15	WITHDRAWAL	15000.0

```
sqlite>
```

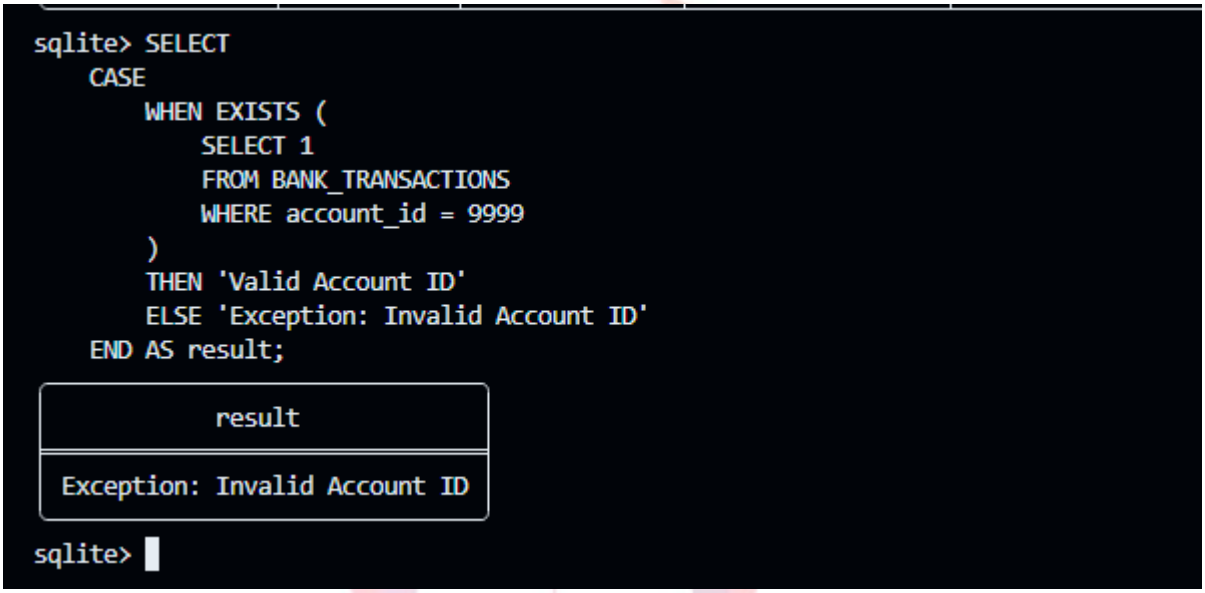
Exception Handling

Case 1 – Invalid Account ID

```
SELECT
  CASE
    WHEN EXISTS (
      SELECT 1
```

```
      FROM BANK_TRANSACTIONS
      WHERE account_id = 9999
    )
    THEN 'Valid Account ID'
    ELSE 'Exception: Invalid Account ID'
  END AS result;
```

Output:

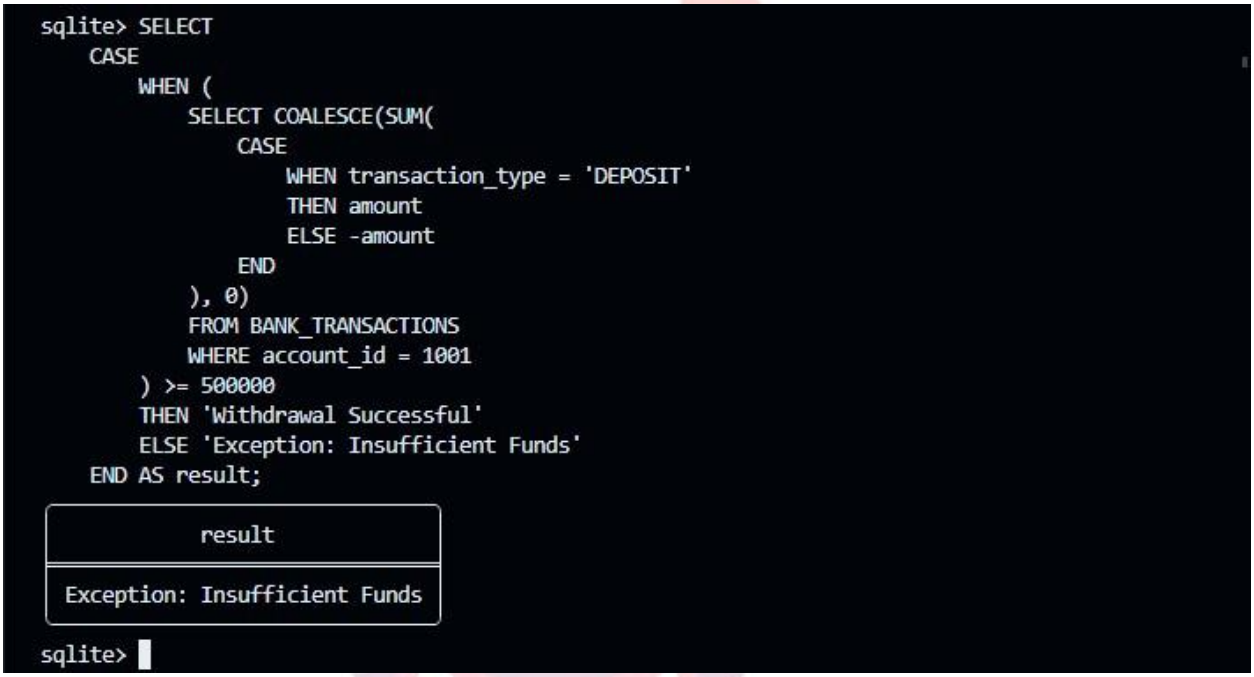


Case 2 – Insufficient Funds

```
SELECT
  CASE
    WHEN (
      SELECT COALESCE(SUM(
        CASE
          WHEN transaction_type = 'DEPOSIT'
          THEN amount
          ELSE -amount
        END
```

```
    ), 0)
    FROM BANK_TRANSACTIONS
    WHERE account_id = 1001
  ) >= 500000
  THEN 'Withdrawal Successful'
  ELSE 'Exception: Insufficient Funds'
END AS result;
```

Output:



Case 3 – Invalid Transaction Amount

```
SELECT
  CASE
    WHEN -500 > 0
    THEN 'Valid Transaction Amount'
    ELSE 'Exception: Invalid Transaction Amount'
  END AS result;
```

Output:

```
sqlite> SELECT
    CASE
        WHEN -500 > 0
        THEN 'Valid Transaction Amount'
        ELSE 'Exception: Invalid Transaction Amount'
    END AS result;
```

result
Exception: Invalid Transaction Amount

```
sqlite>
```

Dataset Insights

- **The dataset contains 25 banking transaction records.**
- **The transactions belong to 5 different bank accounts/customers.**
- **The transaction types include Deposit and Withdrawal.**
- **Transaction dates and amounts are recorded for each banking transaction.**
- **Deposit and withdrawal amounts can be compared to analyze account activity.**
- **Transactions exceeding ₹50,000 can be identified using SQL.**
- **Account-wise aggregation helps analyze the total transaction amount and account activity.**
- **Monthly transaction aggregation helps analyze deposit and withdrawal trends.**

Result

Thus, the Banking Account and Transaction Management system was successfully implemented using SQL and SQLite. The required table was created, 25 transaction records were inserted, and the required SQL queries were executed successfully. Transaction details, total deposits and withdrawals, highest transaction customers, transactions exceeding ₹50,000, and monthly transaction totals were analyzed successfully. SQLite-compatible validation and balance calculation operations were also implemented successfully.